



tecnun

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA
Escuela Superior de Ingenieros · Ingeniarien Goi Mailako Eskola

● C++ Laboratory Lessons

Lesson N° 7



Computer Programming II

Dr. Paul Bustamante

OUTLINE

OUTLINE.....	1
1. OOP: Introduction.....	1
1.1 Exercise 1: My first class complex	1
1.2 Exercise 2: Modulus of a complex.....	2
1.3 Exercise 3: Adding more members to the class complex.....	2
1.4 Exercise 4: The class Point.....	4

1. OOP: Introduction.

In this exercise, you will start using classes to create new objects, and to understand the encapsulation and data abstraction, i.e. everything related to OOP.

1.1 Exercise 1: My first class *complex*

In this exercise, you will create your first class called ***complex***, which can work with complex numbers as if they were another type of variable of C++, analogous to *int*, *long*, *double*, etc.

You will call this Project ***Exer1***. You will have to create three files ***complex.h***, ***complex.cpp*** and ***main.cpp***. These are shown next:

```
// file complex.h

class complex
{
    double real;
    double imag;
public:
    void SetData(void);    // Assign values
    void Prt();            // Output values
};
```

```
// file complex.cpp

#include "complex.h"
#include <iostream>
using namespace std;

void complex::SetData(void) // member function SetData()
{
    cout << "Real part: ";
    cin >> real;
    cout << "Imaginary part: ";
    cin >> imag;
}
void complex::Prt()
{
    cout << real << ", " << imag << endl;
}
```

```
//file main.cpp

#include "complex.h"
void main(void)
{
    complex c1, c2; // Objects of "complex" class
    c1.SetData();
    c2.SetData();
}
```

```

cout << "First complex: "; c1.Prt();
cout << "Second complex: "; c2.Prt();
cout << "I've already finished." << endl;
}

```

Compile the project and run it, observing the result. Create some more complex numbers in the main program, operate with them and print the result.

1.2 Exercise 2: Modulus of a complex.

To better understand classes and objects, we will continue to add more methods to the class `complex`. Now you must modify the files to do the following: create a new **member function** named **`modulus()`** to calculate the modulus of the complex object that is its **implicit argument**.

The prototype of this function inside the class must be: `double modulus(void);`

It will be used, for example, in this way:

```

complex a;
a.SetData();
cout << "The modulus of ";
a.Prt();
cout << " is " << a.modulus() << endl;

```

As you must know, the modulus of a complex number is $\sqrt{\text{real}^2 + \text{imag}^2}$.

Modify the program **`main.cpp`** so that it uses the function **`modulus()`**. This is a good example of the possibilities of C++ in order to define new data types.

1.3 Exercise 3: Adding more members to the class `complex`

In order to preserve what done in the previous exercise, create a new project called **`Exer3`** in your working directory. Copy your source files from Exercise 1, **`complex.h`**, **`complex.cpp`** and **`main.cpp`**. Rename these files calling them **`complex3.h`**, **`complex3.cpp`** and **`main3.cpp`**.

Note the new methods added, and the new code in **`main()`**:

```

//file complex3.h
class complex
{
    double real;
    double imag;
public:
    // SetThings
    void SetData(void);
    void SetReal(double);
    void SetImag(double);
    // GetThings
    double GetReal(void){return real;}
    double GetImag(void){return imag;}

    //Operations
    complex Add(complex& c);
    complex Subtraction(complex& c);
    complex Multiplication(complex& c);
    complex Division(complex& c);
    //Output
    void Prt();
};

```

```

//file complex3.cpp
#include "complex.h"
#include <iostream>
using namespace std;
// member function SetData()
void complex::SetData(void)
{
    cout << "Real part: ";
    cin >> real;
    cout << "Imaginary part: ";
    cin >> imag;
}
void complex::SetReal(double re)
{
    real = re;
}
void complex::SetImag(double im)
{
    imag = im;
}
complex complex::Add(complex& c)
{
    complex cs;
    cs.real = real + c.real;
    cs.imag = imag + c.imag;
    return cs;
}
//Write yourself the Subtraction, Multiplication and Division functions
//Multiplication of complex numbers: (a,b) * (c,d) = (ac-bd, ad+bc)
//Division of complex numbers:
// (a,b)/(c,d)=((ac+bd)/(c^2+d^2),(bc-ad)/(c^2+d^2))

```

```

//file main.cpp
#include "complex3.h"
void main(void)
{
    // complex numbers declarations
    complex c1,c2,c3;
    c1.SetData();
    c2.SetData();
    c3.SetData();           //ask data for c3
    //assign data to c4
    double r,i;
    cout << "Give r,i of c4:"; cin >> r >> i;
    complex c4;
    c4.SetReal(r); c4.SetImag(i);
    // arithmetic operators
    complex add = c1.Add(c2);           //c1+c2
    complex sub = c1.Subtraction(c3);   //c1-c3
    complex mul = c2.Multiplication(c4); //c2*c4
    complex div = c1.Division(c3);      //c1/c3

    cout << "First complex: "; c1.Prt();
    cout << "Real    : " << c1. GetReal();
    cout << "Imagin.: " << c1. GetImag();

    cout << "Second complex: "; c2.Prt();
    cout << "Third complex: "; c3.Prt();
    cout << "Addition: " ;    add.Prt();
    cout << "Subtraction: " ;    sub.Prt();
    cout << "Multiplication: " ;    mul.Prt();
    cout << "Division: " ;    div.Prt();

    cout << "I've already finished." << endl;
}

```

Compile the project and run it, observing the result. Create some more complex numbers in the main program, operate with them and print the result.

1.4 Exercise 4: The class Point

This exercise consists of calculating the **distance** between two or more points, using **OOP**. To do this you should create the class **Point**, which must have two **private double** type variables, **x** and **y** and the necessary functions so that the following sentences or statements can be executed:

```
#include <iostream>
using namespace std;
#include "Point.h"

void main ()
{
    Point p1, p2;          // Create two Point objects

    cout << "P1 -> ";
    p1.AskData();          //Ask to enter data from console

    cout << "P2 -> ";
    p2.AskData();          //Ask to enter data from console

    double dist = p1.Distance(p2);

    cout << "The distance between P1 and P2 is:" << dist << endl;
    cout << "The end..." << endl;
}

//file Point.h
class Point
{
    double x;
    double y;

    public:
        //Add the functions here
};
```